

Trabajo Práctico N° 2

La Lista Negra



Fecha presentación	30/10/2025
Fecha entrega	20/11/2025

1 Introducción

El Bar de Moe es un lugar muy concurrido por todo tipo de individuos. La mayoría solo vienen a pasar un buen rato a solas o con sus amigos, pero otros llegan con ganas de causar problemas. Moe armó una lista con todas las personas que tienen prohibida la entrada (la denominada "lista negra"). Nosotros somos los encargados de cuidar la entrada y decidir quien puede entrar y quien no.

2 Objetivo

El presente trabajo práctico tiene como objetivo que el alumno:

- Se familiarice con el manejo de archivos en C.
- Realice apropiadamente las operaciones con archivos.
- Ponga en práctica el uso de argumentos por línea de comando.
- Se familiarice con y utilice correctamente memoria dinámica.
- Logre utilizar correctamente un TDA externo.

Por supuesto, se requiere que el trabajo cumpla con las **buenas prácticas de programación** profesadas por la cátedra. Se considerarán críticos la modularización, reutilización y claridad del código.

3 Enunciado

El objetivo del trabajo es crear un juego de memoria que simule el trabajo del guardia del Bar de Moe. Antes de comenzar el juego, se deberá entregar una lista de personas que no pueden entrar, el jugador deberá memorizar esta lista y no podrá consultarla más adelante. Durante el transcurso del juego, el jugador irá recibiendo nombres y tendrá que decidir si puede pasar o no, recordando los nombres de la lista negra. Cada respuesta correcta sumará puntos, el juego terminará cuando el jugador deje pasar a alguien de la lista negra y su puntaje será anotado en el ranking.

3.1 Argumentos

El programa debe ser llamado con un argumento:

```
1 ./lista_negra <archivo_lista_negra> <archivo_ranking>
```

Si el programa se llamó con la cantidad incorrecta de argumentos debe devolver un error.

3.1.1 Archivo de Lista Negra

El archivo de entrada será la lista negra que contiene las personas que no pueden entrar en el bar. Será un archivo de CSV con el siguiente formato:

```
1 <nombre>;<apellido>
```

Se sabe que tanto el nombre como el apellido no van a contener más de 100 caracteres cada uno y que ninguno va a contener el caracter ';'. Se puede asumir que este archivo es pequeño y se deberá guardar la lista de nombres en memoria. No se cuenta con una cota máxima para el largo de la lista, para guardarse en memoria tendrá que hacerse con la cantidad justa de bytes.

En caso de que el archivo esté vacío o no exista, se debe devolver un error.

3.1.2 Archivo de Ranking

El archivo de ranking contiene un historial de todas las partidas. Si está vacío se puede asumir que es la primera partida, y en caso de no existir, se debe crear uno nuevo. Será de tipo CSV y tendrá el siguiente formato:

```
1 <nombre_jugador>;<largo_lista_negra>;<puntaje>
```

- `nombre_jugador` : Tiene como máximo 500 caracteres, se puede asumir que no contiene el caracter ‘,’.
- `largo_lista_negra` : De tipo entero, contiene la cantidad de nombres que había en la lista negra en esa partida.
- `puntaje` : De tipo entero, contiene el puntaje de la partida.

Este archivo puede ser muy grande (ya que contiene datos históricos de todas las partidas) así que no puede ser guardado completamente en memoria.

El archivo deberá estar ordenado por puntaje. En caso de tener igual puntaje, el que tenga mayor largo de lista estará primero. En caso de tener igual puntaje y lista, será por orden alfabético del nombre. El orden del ranking se debe validar antes de empezar con el juego, en caso de que el orden sea incorrecto se debe devolver un error.

3.2 Desarrollo del Juego

Antes de comenzar, se le mostrará al jugador la lista de personas **que no pueden entrar al bar** y se le pedirá que ingrese su nombre (para ser usado en el ranking al finalizar). Luego de ingresado el nombre, se limpiará la pantalla y se dará inicio al juego.

Se le mostrarán nombres al azar al jugador y en cada uno se le preguntará si está admitido o no. Ingresará 'S' para si y 'N' para no. Hay cuatro posibles resultados:

- **La persona NO estaba en la lista negra y el jugador SI lo admitió:** En este caso el jugador respondió correctamente y suma 1 punto.
- **La persona NO estaba en la lista negra y el jugador NO lo admitió:** En este caso el jugador cometió un error, pero no es tan grave, se le restará 1 punto.
- **La persona SI estaba en la lista negra y el jugador NO lo admitió:** En este caso el jugador respondió correctamente y evitó un posible problema en el bar, suma 3 puntos.
- **La persona SI estaba en la lista negra y el jugador SI lo admitió:** En este caso el jugador dejó pasar a un cliente problemático, el juego deberá finalizar.

Al finalizar, se le mostrará el puntaje final junto con su puesto en el ranking. Además, el archivo de ranking debe ser debidamente actualizado.

3.3 TDA Generador de nombres

Para la ejecución del juego se contará con un TDA generador de nombres que **ya estará implementado por la cátedra** y deberá ser utilizado (el código no va a estar disponible).

Las primitivas del TDA son las siguientes:

```

1 #ifndef __GENERADOR_H__
2 #define __GENERADOR_H__
3
4 #include <stdbool.h>
5
6 #define MAX_NOMBRE 100
7 #define MAX_APELLIDO 100
8
9 typedef struct persona {
10     char nombre[MAX_NOMBRE];
11     char apellido[MAX_APELLIDO];
12 } persona_t;
13
14 typedef struct generador generador_t;
15
16 // Pre: -
17 //
18 // Post: Crea un TDA generador en el heap y devuelve un puntero
19 // al mismo. Devuelve NULL si no lo pudo crear.
20 generador_t *generador_crear();
21
22 // Pre: `generador` fue previamente creado con `generar_crear`.

```

```

23 //
24 // Post: Agrega una persona nueva a la lista negra del generador. A partir de
25 // que es agregada, tiene una chance de ser devuelta por el generador de nombres.
26 // Devuelve `true` si se pudo agregar o `false` si falló.
27 bool agregar_persona_lista_negra(generator_t *generador, persona_t persona);
28
29 // Pre: `generador` fue previamente creado con `generador_crear`.
30 //
31 // Post: Devuelve una persona aleatoria, la misma puede ser parte de la lista
32 // negra o ser una persona con un nombre aleatoriamente generado.
33 persona_t persona_aleatoria(generator_t *generador);
34
35 // Pre: El generador recibido fue previamente creado con `generador_crear`.
36 // Post: Libera la memoria que se reservó en el heap para el generador.
37 void generador_destruir(generator_t *generador);
38
39 #endif /* __GENERADOR_H__ */

```

El TDA generador se usará para generar los nombres aleatorios que se le mostraran al jugador. No hace falta que el alumno decida cuando mostrar una persona de la lista negra y cuando uno admitido, esa lógica ya está contemplada internamente en el TDA.

Cada vez que se llame a `persona_aleatoria`, el generador devolverá una persona aleatoria; pero a veces, decidirá devolver a alguien de la lista negra. Es por eso que, para que el TDA funcione correctamente, el alumno deberá inicializarlo con todas las personas de la lista negra recibida por argumento. Esto se hace con `agregar_persona_lista_negra`.

El alumno debe encargarse de mostrarle correctamente la persona al jugador y preguntarle si puede pasar o no. Luego de obtener una respuesta, el alumno tendrá que encargarse de ver si es correcta o no, buscando el nombre generado en la lista negra. **El TDA no valida las respuestas del jugador, solo devuelve nombres.**

4 Resultado esperado

Esperamos que el trabajo cumpla la funcionalidad explicada anteriormente. Se deberá:

- Implementar el desarrollo del juego tal cual se describe.
- Utilizar correctamente el TDA provisto por la cátedra.
- Utilizar correctamente memoria dinámica, es decir, no tener pérdidas de memoria al ejecutarse el programa.
- Respetar las buenas prácticas de programación que profesamos en la cátedra.

5 Compilación y Entrega

El trabajo práctico debe ser realizado en un archivo llamado `lista_negra.c`

El trabajo debe ser compilado sin errores al correr el siguiente comando:

```
1 gcc *.c generador.o -o lista_negra -std=c99 -Wall -Wconversion -Werror -lm
```

Por último debe ser entregado en la plataforma de corrección de trabajos prácticos **AlgoTrón** (patente pendiente), en la cual deberá tener la etiqueta **¡Exito!** significando que ha pasado las pruebas a las que la cátedra someterá al trabajo.

IMPORTANTE! *Esto no implica necesariamente haber aprobado el trabajo ya que además será corregido por un colaborador que verificará que se cumplan las buenas prácticas de programación. Para que el trabajo sea corregido por un colaborador, el mismo debe pasar todas las pruebas mínimas..*

Para la entrega en **AlgoTrón** (patente pendiente), recuerde que deberá subir un archivo **zip** conteniendo únicamente los archivos antes mencionados, sin carpetas internas ni otros archivos. De lo contrario, la entrega no será validada por la plataforma.

6 Anexo

6.1 FAQ

En este [link](#) encontrarán el documento de FAQ del TP, donde se irán cargando dudas realizadas con sus respuestas. Todo lo que esté en ese documento es válido y oficial para la realización del TP.